

Contrato de interfaz: Pagos ↔ Entradas/Inventario

Versión: 2.1 (API v1) — 28/09/2026. Respuesta de Pagos a la v2.0 (flujo asíncrono) de Entradas/Inventario.

Equipo consumidor: Entradas/Inventario

Equipo proveedor: Pagos

Basado en: Contrato Entradas → Pagos v2.0 “Flujo asíncrono” (Sebastián Fuentes); HU 1.3 (#4), HU 1.6 (#8), HU 3.1 (#16) y HU 3.3 (#18); ítems de integración INT-02 (#30) e INT-03 (#31); tabla de dependencias TITEC 2026-2 (fila Pagos) y Swagger docs/openapi.yaml, tag "Entradas (consumidor)".

1. Propósito

Entradas necesita cobrar las entradas de una reserva y conocer el resultado del cobro para emitir la entrada con QR o liberar el stock. El cobro no se resuelve en una sola petición: Pagos crea el pago en estado PENDIENTE y entrega una `url_checkout` donde el comprador ingresa su tarjeta (directo contra Stripe). Hoy el resultado se conoce consultando el estado del pago (operación 3); la publicación de eventos en el broker está pendiente (sección 8). Pagos también permite anular o reembolsar el pago si la reserva vence o se queda sin stock.

2. Operación: Crear intención de pago

2.1 Descripción

Registra el cobro de una reserva: crea el pago en estado PENDIENTE en Stripe y devuelve el `id_pago` y la `url_checkout` a la que Entradas redirige al comprador.

2.2 Quién la expone

Equipo Pagos.

2.3 Quién la consume

Equipo Entradas/Inventario, en el momento en que el comprador confirma su reserva y elige pagar (la reserva ya tiene stock apartado).

2.4 Endpoint

POST `/api/v1/pagos/transacciones`

Authorization: Bearer <JWT de Auth> (o el mismo JWT en `token_sesion`, en el body)

2.5 Request (lo que se envía)

Campo	Tipo	Obligatorio	Descripción
id_reserva	string (máx. 64)	Sí	Id de la reserva en Entradas. Es la llave de idempotencia: un pago por reserva.
id_evento	string (máx. 64)	No	Opcional. Id del evento para describir el cobro; si no viene, Pagos lo toma de la reserva de Entradas. No se almacena.
total	integer	Sí	Monto total en CLP, sin decimales (1 a 99.999.999).
cantidad_entradas	integer	Sí	Cantidad de entradas de la reserva (1 a 100). Se valida contra Entradas; no se almacena.
metodo	string	No	"formulario" (por defecto, pantalla de pago de Pagos) o "checkout" (página alojada por Stripe).
token_sesion	string (JWT)	No	JWT del comprador emitido por Auth. Alternativa al header Authorization (compatibilidad con la v1.0); si vienen ambos, se usa el header.

Ejemplo:

```
{
  "id_reserva": "res-98765",
  "total": 15000,
  "cantidad_entradas": 2,
  "token_sesion": "eyJhbGciOiJIUzI1NiIsInR5cCI6I.."}
}
```

2.6 Response (lo que se recibe)

Campo	Tipo	Obligatorio	Descripción
id_pago	string (UUID)	Sí	Identificador del pago. Entradas lo guarda como referencia.
id_reserva	string	Sí	Eco de la reserva recibida.
estado_pago	string	Sí	PENDIENTE al crear (estados posibles en la sección 3.6).
fecha_creacion	string (ISO 8601)	Sí	Fecha de creación de la compra.
url_checkout	string	Sí	URL a la que Entradas redirige al comprador para pagar. null si el pago ya no está PENDIENTE.

Campo	Tipo	Obligatorio	Descripción
metodo	string	Sí	Método de pago usado: formulario o checkout.
fecha_expiracion	string (ISO 8601)	No	Hasta cuándo es válido el enlace de pago.
client_secret	string	No	Uso interno del front de Pagos (Stripe Elements). Entradas debe ignorarlo y no almacenarlo.

Ejemplo:

```
{
  "id_pago": "3f2b8c1e-6a4d-4e2f-9b1a-7c5d8e9f0a12",
  "id_reserva": "res-98765",
  "estado_pago": "PENDIENTE",
  "fecha_creacion": "2026-09-30T19:40:00.000Z",
  "url_checkout": "http://localhost:5173/?id_pago=3f2b8c1e-6a4d-4e2f-9b1a-7c5d8e9f0a12",
  "metodo": "formulario",
  "fecha_expiracion": "2026-09-30T20:11:00.000Z"
}
```

Nota de diseño: Pagos no devuelve ni recibe datos de la tarjeta (van directo del navegador a Stripe) ni datos personales del comprador: el id_usuario se toma del token. Entradas no replica el pago en su base de datos; solo guarda id_pago como referencia.

2.7 Códigos de error

Código	Significado
200	No es error: reintento idempotente con el mismo id_reserva y los mismos datos. Devuelve el pago ya existente, sin crear un segundo cobro.
400	VALIDACION_FALLIDA o JSON_INVALIDO: faltan campos o tienen formato inválido (se listan todos en "detalles").
401	NO_AUTENTICADO: no viene el token (ni en el header ni en token_sesion) o es inválido/expirado.
409	CONFLICTO_IDEMPOTENCIA: la reserva ya tiene un pago con datos distintos (ej. otro total).
422	ORDEN_INCONSISTENTE (total o cantidad no coinciden con la reserva en Entradas) o RESERVA_NO_ENCONTRADA.
502	ERROR_PASARELA: Stripe rechazó la operación o no respondió a tiempo.
503	ENTRADAS_NO_DISPONIBLE: Pagos no pudo verificar la orden con el servicio de Entradas.

Código	Significado
500	ERROR_INTERNO: error interno del servicio.

2.8 Tiempo de respuesta esperado (SLA)

< 2000 ms (incluye la creación del cobro en Stripe; la verificación contra Entradas corta a los 500 ms).

3. Operación: Consultar estado del pago

3.1 Descripción

Devuelve el estado actual de un pago; si sigue PENDIENTE, Pagos lo confirma además con Stripe. Mientras Pagos no publique eventos en el broker, es la forma de conocer el resultado del cobro.

3.2 Quién la expone

Equipo Pagos.

3.3 Quién la consume

Equipo Entradas/Inventario, después de redirigir al comprador, consultando cada pocos segundos (polling) hasta obtener un estado final.

3.4 Endpoint

GET /api/v1/pagos/{id_pago}

Authorization: Bearer <JWT de Auth: rol SERVICIO o el comprador dueño del pago>

3.5 Request (lo que se envía)

Campo	Tipo	Obligatorio	Descripción
id_pago	string (UUID), en la ruta	Sí	Id entregado por la operación 2 (Crear intención de pago).

Ejemplo:

GET /api/v1/pagos/3f2b8c1e-6a4d-4e2f-9b1a-7c5d8e9f0a12

3.6 Response (lo que se recibe)

Campo	Tipo	Obligatorio	Descripción
id_pago	string (UUID)	Sí	Identificador del pago.
id_reserva	string	Sí	Reserva a la que corresponde el pago.

Campo	Tipo	Obligatorio	Descripción
estado_pago	string	Sí	PENDIENTE, APROBADO, RECHAZADO, ANULADO o REEMBOLSADO.
fecha_creacion	string (ISO 8601)	Sí	Fecha de creación de la compra.
monto	integer	Sí	Monto cobrado en CLP.
moneda	string	Sí	Siempre "CLP".
motivo_rechazo	string	No	Motivo cuando el estado es RECHAZADO (ej. sesión expirada).
fecha_actualizacion	string (ISO 8601)	Sí	Último cambio de estado.
url_checkout	string	No	Solo mientras el pago está PENDIENTE.

Ejemplo:

```
{
  "id_pago": "3f2b8c1e-6a4d-4e2f-9b1a-7c5d8e9f0a12",
  "id_reserva": "res-98765",
  "estado_pago": "APROBADO",
  "monto": 15000,
  "moneda": "CLP",
  "motivo_rechazo": null,
  "url_checkout": null,
  "fecha_creacion": "2026-09-30T19:40:00.000Z",
  "fecha_actualizacion": "2026-09-30T19:42:10.000Z"
}
```

Nota de diseño: Estados finales: APROBADO, RECHAZADO, ANULADO y REEMBOLSADO. Un pago APROBADO nunca vuelve a PENDIENTE ni a RECHAZADO. La respuesta no incluye datos de la tarjeta ni del comprador más allá de su id.

3.7 Códigos de error

Código	Significado
400	VALIDACION_FALLIDA: el id_pago no es un UUID válido.
401	NO_AUTENTICADO: falta el token o es inválido.
403	NO_AUTORIZADO: el pago es de otro usuario y el token no tiene rol SERVICIO/ADMIN.
404	PAGO_NO_ENCONTRADO: no existe un pago con ese id.
500	ERROR_INTERNO: error interno del servicio.

3.8 Tiempo de respuesta esperado (SLA)

Propuesta: < 300 ms cuando el estado ya es final; < 2000 ms si Pagos debe confirmar con Stripe un pago PENDIENTE.

4. Operación: Anular o reembolsar un pago

4.1 Descripción

Revierte un pago cuando Entradas no emitirá la entrada: si está PENDIENTE lo anula sin cobrar; si está APROBADO devuelve el total al comprador.

4.2 Quién la expone

Equipo Pagos.

4.3 Quién la consume

Equipo Entradas/Inventario, en el momento en que la reserva vence (hold vencido) o se queda sin stock y el pago ya fue creado.

4.4 Endpoint

POST /api/v1/pagos/{id_pago}/reembolso

Authorization: Bearer <JWT de Auth, rol SERVICIO>

4.5 Request (lo que se envía)

Campo	Tipo	Obligatorio	Descripción
id_pago	string (UUID), en la ruta	Sí	Pago que se debe anular o reembolsar.
motivo	string (3 a 255)	Sí	Razón de la reversión; queda registrada en Pagos.

Ejemplo:

```
{
  "motivo": "Hold de la reserva vencido: el usuario pagó fuera de plazo"
}
```

4.6 Response (lo que se recibe)

Campo	Tipo	Obligatorio	Descripción
id_pago	string (UUID)	Sí	Pago revertido.
estado_pago	string	Sí	ANULADO (no hubo cobro) o REEMBOLSADO (se devolvió el dinero).

Campo	Tipo	Obligatorio	Descripción
accion	string	Sí	ANULADO o REEMBOLSADO: lo que hizo Pagos.
ya_procesado	boolean	Sí	true si el pago ya estaba anulado o reembolsado (reintento).
reembolso	objeto	No	Solo si se reembolsó: id_reembolso, monto, motivo, estado_reembolso y fecha_creacion.
fecha_actualizacion	string (ISO 8601)	Sí	Momento del cambio de estado.

Ejemplo:

```
{
  "id_pago": "3f2b8c1e-6a4d-4e2f-9b1a-7c5d8e9f0a12",
  "estado_pago": "REEMBOLSADO",
  "accion": "REEMBOLSADO",
  "ya_procesado": false,
  "reembolso": {
    "id_reembolso": "8a1c2d3e-4f5a-4b6c-9d7e-0f1a2b3c4d5e",
    "monto": 15000,
    "motivo": "Hold de la reserva vencido",
    "estado_reembolso": "EXITOSO",
    "fecha_creacion": "2026-09-30T20:15:00.000Z"
  },
  "fecha_actualizacion": "2026-09-30T20:15:00.000Z"
}
```

Nota de diseño: Siempre es un reembolso total; el reembolso parcial está fuera del alcance de esta entrega. Entradas no necesita saber si el pago estaba PENDIENTE o APROBADO: Pagos decide si anula o reembolsa (incluso si el comprador pagó justo antes de la anulación).

4.7 Códigos de error

Código	Significado
200	No es error: si el pago ya estaba ANULADO o REEMBOLSADO se responde ya_procesado: true (no se devuelve dos veces).
400	VALIDACION_FALLIDA: falta el motivo o el id_pago no es un UUID.
401	NO_AUTENTICADO: falta el token o es inválido.
403	NO_AUTORIZADO: el pago es de otro usuario y el token no tiene rol SERVICIO/ADMIN.
404	PAGO_NO_ENCONTRADO: no existe el pago.
409	PAGO_NO_REEMBOLSABLE (el pago está RECHAZADO: no hubo cobro) o CAMBIO_CONCURRENTE (cambió de estado en paralelo; reintentar).

Código	Significado
502	ERROR_PASARELA: Stripe no pudo procesar la anulación o el reembolso.
500	ERROR_INTERNO: error interno del servicio.

4.8 Tiempo de respuesta esperado (SLA)

Propuesta: < 2000 ms (requiere una llamada a Stripe).

5. Reglas de uso (lado consumidor)

Condiciones que sigue Entradas/Inventario al usar estas operaciones.

1. Crear el pago con el JWT del comprador (header Authorization o token_sesion en el body). Para consultar y anular, usar el header Authorization con un JWT de rol SERVICIO o el del mismo comprador.
2. Llamar a "Crear intención de pago" una sola vez por reserva. Si hay timeout o error de red, reintentar con el mismo id_reserva y los mismos datos: la operación es idempotente y no genera un segundo cobro.
3. No cambiar el total ni la cantidad de una reserva que ya tiene pago: Pagos responde 409. Si la orden cambia, anular el pago y usar una reserva nueva.
4. La creación del pago responde PENDIENTE (igual que en la v2.0). Redirigir al comprador a la url_checkout recibida: ahí ingresa su tarjeta. Entradas no pide ni recibe datos de tarjeta.
5. Consultar "Consultar estado del pago" cada 5 segundos (propuesta) hasta un estado final. El GET no lleva body: el token va en el header Authorization.
6. Emitir la entrada con QR solo cuando estado_pago = APROBADO (equivale a "PagoAprobado" de la v2.0). RECHAZADO (equivale a "PagoRechazado") significa que no hubo cobro y la reserva puede liberarse.
7. Un pago puede seguir PENDIENTE más que el TTL de la reserva (12 a 15 min en la v2.0). Si la reserva vence o se queda sin stock, llamar a "Anular o reembolsar un pago" con un motivo; Pagos anula, o reembolsa si el comprador alcanzó a pagar.
8. Guardar solo el id_pago como referencia; no copiar el monto ni el estado del pago como tabla propia.
9. Manejar los errores por el campo "error" del cuerpo ({ "error": "CODIGO", "mensaje": "...", "detalles": [...] }), no por el texto del mensaje.

6. Versionado y cambios

- Cualquier cambio en el request/response debe ser **versionado** (ej. v1, v2) y comunicado con anticipación.

- Cambios que rompan compatibilidad (breaking changes) requieren período de transición acordado entre ambos equipos.
- Versión vigente: v1 (prefijo /api/v1). La especificación completa está en el Swagger del módulo Pagos: docs/openapi.yaml (servido en /api-docs).

7. Dueños del contrato

Rol	Equipo	Contacto
Proveedor	Pagos	Simón Canales (Scrum Master)
Consumidor principal	Entradas/Inventario	Sebastián Fuentes (Scrum Master)

8. Diferencias con la v2.0 de Entradas y pendientes

- ☐ Coincide con la v2.0: POST /api/v1/pagos/transacciones con id_reserva, total, cantidad_entradas y token_sesion; respuesta con id_pago y estado_pago = PENDIENTE; SLA < 2000 ms solo para la creación; GET /api/v1/pagos/{id_pago} para conocer el estado.
- ☐ La respuesta de creación trae además fecha_creacion (fecha de creación de la compra, tabla TITEC) y url_checkout. Entradas debe redirigir al comprador a url_checkout para que pague.
- ☐ Eventos “PagoAprobado” / “PagoRechazado” por el broker: Pagos todavía no los publica. Pendiente coordinar con el ayudante (equipo plataforma) la conexión, el nombre del exchange/cola y el formato del evento. Mientras tanto, el resultado se obtiene con GET /api/v1/pagos/{id_pago}.
- ☐ Tiempo del pago: puede seguir PENDIENTE más de 10 minutos (una sesión de Stripe dura mínimo 30 min; el formulario no expira solo). Si vence la reserva, Entradas llama a POST /api/v1/pagos/{id_pago}/reembolso (operación 4).
- ☐ Códigos de error: además de 400 y 500, Pagos responde 401, 409, 422, 502 (pasarela) y 503 (sección 2.7). id_pago es un UUID.
- ☐ Reintentos: Entradas puede reintentar la creación con el mismo id_reserva sin riesgo (no se cobra dos veces).
- ☐ Por confirmar: intervalo de consulta (propuesta 5 s) y SLA de las operaciones 3 y 4.
- ☐ Por confirmar: ruta del servicio de Entradas que Pagos invoca para verificar la orden: GET /api/v1/entradas/reservas/{id_reserva} (INT-01, #29).